

ASE 420 Tools Installation - Git and GitHub

Git and GitHub

- **Git** records changes to files on your computer.
- **GitHub** hosts Git repositories so a team can share and review work.
- ASE 420 teams use them to coordinate code and preserve project history.

Install and Check Git

- **Windows:** download the installer from git-scm.com.
- **macOS:** `brew install git`, or download from git-scm.com.
- **Linux:** `sudo apt install git` (Debian/Ubuntu) or your distro's package manager.
- Create a free [GitHub account](https://github.com) if you don't already have one.
- Check the installation in a terminal:

Basic Configuration

- Use a name and email associated with your GitHub account, so commits are linked to your profile.
- The email doesn't have to be your public email address — a verified email on your GitHub account, or GitHub's private `noreply` address (Settings → Emails), also works.

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
git config --global init.defaultBranch main
```

Get a Repository

- Copy the repository URL from GitHub.
- Clone it once, then work inside the new directory.

```
git clone https://github.com/nkuase/ase420.git  
cd ase420  
git status
```

GitHub Authentication for Push

- Cloning a **public** repository works without signing in, but **pushing** requires GitHub authentication.
- In VS Code, sign in to GitHub when prompted (the Accounts icon, or `Git: ...` in the Command Palette) — the simplest option for most students.
- Alternatives: a Git Credential Manager (bundled with recent Git installers), or an SSH key.
- If `git push` fails with a permission or

The Basic Git Cycle

1. Edit files.
2. Inspect the changes.
3. Stage the intended files.
4. Commit one meaningful change.
5. Push the commit to GitHub.

```
git status  
git add path/to/file.py  
git commit -m "Add collision detection"  
git push
```

Understand `status`, `add`, and `commit`

- `git status` shows changed, staged, and untracked files.
- `git add` chooses what will go into the next commit.
- `git commit` records the staged snapshot locally.
- A commit does **not** reach GitHub until you push it.

Pull Before You Work

- Update `main` before creating a new branch, so your work starts from the latest code.

```
git switch main  
git pull
```

- To sync an existing working branch with new remote changes, follow your team's merge or rebase convention — don't pull into a branch with uncommitted work without checking first.
- Read conflicts carefully; do not blindly discard either version.

Repository Workflow for ASE 420

- Begin from an updated `main` branch.
- Create a short-lived branch for one task.
- Commit small, working changes with clear messages.
- Push the branch and open a pull request on GitHub.
- Ask a teammate to review before merging.

This branch → commit → push → pull request

Example Branch Workflow

```
git switch main
git pull
git switch -c feature/next-piece

# Edit and test your files

git status
git diff
git add path/to/file.py
git diff --staged
git commit -m "Add next-piece preview"
git push -u origin feature/next-piece
```

After the Pull Request

- **Push** does not merge your code — it only uploads the branch. Merging happens through the pull request on GitHub.
- A teammate **reviews** the pull request and **merges** it once it looks good.
- Back on your machine: `git switch main`, then `git pull` to get the merged change.
- Delete the finished branch locally and on GitHub to keep the branch list clean.

```
git switch main
```

A Good Commit

- Has one clear purpose.
- Uses a short action-oriented message.
- Does not include unrelated generated or personal files.

```
Add score calculation  
Fix rotation near left wall  
Document setup steps
```

Check Before Every Commit

- Run `git status` and review the files you staged.
- Use `git diff` to review unstaged changes, and `git diff --staged` to review exactly what will be committed — `git status` alone only shows file names, not content.
- Test the program before pushing.
- Prefer `git add path/to/file` over staging everything automatically.
- Keep `.gitignore` updated for `.venv/`,

Common Cautions

- Never commit passwords, API keys, tokens, or private data.
- Do not commit `.venv/`, `__pycache__/`, or large generated files.
- Do not force-push or rewrite shared history without team agreement.
- Do not work directly on `main` when the team uses pull requests.
- Communicate before resolving a conflict you do not understand.

When Something Goes Wrong

- Stop and run `git status` before trying more commands.
- Read the complete error message.
- Ask a teammate before using destructive commands.
- **Committed** work is usually recoverable, even after mistakes; **uncommitted** work may not be. Commit small, working changes often.